

LÓGICA, VARIÁVEIS & OPERADORES



dados → **regras** → **resultado**

Transformar um problema em passos claros usando dados, expressões e operadores.

- **Algoritmo:** sequência finita e ordenada de passos
- **Variável:** nome que referencia um valor na memória
- **Tipos:** inteiro, real, lógico, caractere e texto
- **Atribuição:** $x \leftarrow \text{expressão}$
- **Aritméticos:** +, -, x, ÷, mod
- **Relacionais:** =, ≠, <, >, ≤, ≥
- **Lógicos:** NÃO, E, OU
- **Prioridade:** parênteses → aritméticos → relacionais → lógicos

Dica: Antes de programar, escreva entradas, processamento e saídas. Um algoritmo claro costuma virar código claro.

DECISÕES & ESTRUTURAS DE CONTROLE



SE • SENÃO • CASO

Escolher caminhos diferentes conforme condições verdadeiras ou falsas.

- **SE:** executa um bloco quando a condição é verdadeira
- **SE/SENÃO:** escolhe entre dois caminhos
- **Encadeado:** use quando há várias faixas ou condições
- **ESCOLHA/CASO:** útil para valores discretos de uma variável
- **Condição composta:** combine testes com E / OU
- **Curto-circuito:** a avaliação pode parar quando o resultado já é conhecido
- **Validação:** trate entradas

Dica: Teste mentalmente os casos de fronteira: igual, menor, maior, zero, vazio e valores extremos. É onde muitos erros aparecem.

REPETIÇÃO & RASTREAMENTO



inicializa → **testa** → **atualiza**

Repetir instruções de forma controlada e acompanhar a evolução das variáveis.

- **PARA:** ideal quando o número de repetições é conhecido
- **ENQUANTO:** testa a condição antes de executar
- **REPITA:** executa ao menos uma vez e testa no final
- **Contador:** $i \leftarrow i + 1$
- **Acumulador:** $\text{soma} \leftarrow \text{soma} + \text{valor}$
- **Sentinela:** valor especial encerra a leitura
- **Loop infinito:** condição nunca se torna falsa
- **Rastreamento:** monte tabela com iteração, variáveis e saída

Dica: Em todo laço, identifique três coisas: inicialização, condição de parada e atualização. Se uma faltar, suspeite de erro.

FUNÇÕES & MODULARIZAÇÃO



dividir • **testar** • **reutilizar**

Dividir problemas em partes menores, reutilizáveis e fáceis de testar.

- **Função:** recebe parâmetros e pode retornar um valor
- **Procedimento:** executa uma tarefa sem retorno obrigatório
- **Parâmetro:** dado enviado para a rotina
- **Escopo:** define onde uma variável pode ser acessada
- **Vetor:** coleção indexada de elementos do mesmo tipo
- **Índice:** posição usada para acessar vetor[i]
- **Matriz:** estrutura bidimensional linha x coluna

Dica: Dê a cada função uma responsabilidade principal. Funções pequenas e bem nomeadas são mais fáceis de entender, testar e reutilizar.